

SCHOOL OF CHEMICAL & MATERIALS ENGINEERING
NATIONAL UNIVERSITY OF SCIENCES & TECHNOLOGY, ISLAMABAD

Paper Type: Final Term Exam **Time Allowed:** 3 hrs **Total Marks:** 100

Full Name: _____ **Reg #:** _____

Instructions: Attempt all questions. Write neatly and clearly in the provided space.

Section A - Multiple Choice Questions ($30 \times 1/2 = 15$ Marks)

1. Which of the following correctly creates a dictionary in Python?

- A. {1,2,3}
- B. [("a",1), ("b",2)]
- C. {"a":1, "b":2}
- D. ("a":1,"b":2)

2. What is the output of the following?

```
a = {1, 2, 3}
a.add(2)
print(len(a))
```

- A. 2
- B. 3
- C. 4
- D. Error

3. Which NumPy function creates an array of zeros?

- A. np.zeros()
- B. np.empty()
- C. np.zero()
- D. np.null()

4. What does the following print?

```
import numpy as np
a = np.arange(5)
print(a[2:])
```

- A. [2,3]
- B. [2,3,4]
- C. [3,4]
- D. Error

5. Which function reads a CSV file in pandas?

- A. pd.open()

- B. pd.readfile()
- C. pd.read_csv()
- D. pd.loadcsv()

6. What is the output?

```
s = "hello"
print(s[1:4])
```

- A. "ell"
- B. "hel"
- C. "llo"
- D. Error

7. File mode "a" is used for:

- A. Reading only
- B. Writing and overwriting
- C. Appending to a file
- D. Creating binary files

8. What does this code do?

```
try:
    x = 1/0
except:
    print("Error")
```

- A. Prints nothing
- B. Prints Error
- C. Crashes program
- D. Prints 0

9. Which of the following is a mutable type?

- A. Tuple
- B. List
- C. String
- D. Integer

10. What is the output?

```
d = {"a":1}
print("b" in d)
```

- A. True
- B. False
- C. None
- D. Error

11. What does `set()` produce when used with an empty argument?

- A. `{}` (empty dict)
- B. `()` (empty tuple)
- C. `set()` (empty set)
- D. Error

12. Which of the following drops a column in pandas?

- A. `df.remove("col")`
- B. `df.delete("col")`
- C. `df.drop("col", axis=1)`
- D. `df.cut("col")`

13. Output?

```
import numpy as np
a = np.array([1,2,3])
print(a*2)
```

- A. `[1,2,3,1,2,3]`
- B. `[2,4,6]`
- C. Error
- D. `[1,4,9]`

14. In string methods, `s.upper()`:

- A. Modifies the original string
- B. Returns a new string in uppercase
- C. Works only for alphabets
- D. Throws error for empty strings

15. Which exception occurs when accessing a missing list index?

- A. `KeyError`
- B. `TypeError`
- C. `IndexError`
- D. `AttributeError`

16. What does this code return?

```
d = {"x":10,"y":20}
print(d.get("z", 0))
```

- A. 10
- B. 20
- C. 0
- D. Error

17. Pandas slicing `df.iloc[0:2]` returns:

- A. Rows 1–2
- B. Rows 0–1
- C. Only row 2
- D. Only row 1

18. What is the output?

```
print("abc".replace("b","xy"))
```

- A. "abc"
- B. "axy"
- C. "axyc"
- D. "axyc"

19. Opening a file with "w" mode:

- A. Appends to file
- B. Deletes the file if exists
- C. Opens for read only
- D. Opens for exclusive create

20. What is printed?

```
s = {1,2,3}
t = {3,4}
print(s & t)
```

- A. `{1,2,3,4}`
- B. `{3}`
- C. `{1,2}`
- D. Error

21. What does the `fit()` method do in Linear-Regression?

- A. Splits data into train and test sets
- B. Computes slope and intercept from training data
- C. Standardizes the predictors
- D. Predicts new values

22. After fitting a LinearRegression model as `model`, how do you access the slope?

- A. `model.slope_`
- B. `model.m`
- C. `model.coef_`

- D. `model.weight_`
23. What is stored in `model.intercept_` after training?
- A. The predicted values
 - B. The error term
 - C. The y-intercept of the regression line
 - D. The mean of the dataset
24. Which function is commonly used to split training and testing data?
- A. `train_test()`
 - B. `test_train_split()`
 - C. `train_test_split()`
 - D. `split_data()`
25. Which metric measures the proportion of variance explained by the model?
- A. RMSE
 - B. MAE
 - C. R-squared (R^2)
 - D. MAPE
26. In scikit-learn, which class is used for simple linear regression?
- A. `sklearn.linear_model.LinearRegression`
 - B. `sklearn.linear_model.LogisticRegress`
 - C. `sklearn.linear_model.Lasso`
 - D. `sklearn.linear_model.Ridge`
27. What shape must the feature input `X` have for sklearn `LinearRegression`?
- A. Always a 1D list
 - B. A 2D array-like: $(n_samples, n_features)$
 - C. A dictionary of lists
 - D. Any shape; sklearn converts it automatically
28. What happens if you pass `[1,2,3,4]` directly to `model.fit()` as `X`?
- A. It works perfectly
 - B. It raises a syntax error
 - C. sklearn reshapes it automatically
 - D. It causes a “expected 2D array” error
29. In multiple linear regression, `model.coef_` contains:
- A. Only one coefficient
 - B. The coefficients for every predictor variable
 - C. The intercept and slope together
 - D. Only categorical variable weights
30. What is the correct way to make predictions with a trained model?
- A. `model.guess(X)`
 - B. `model.predict(X)`
 - C. `model.output(X)`
 - D. `model.estimate(X)`

— End of Section —

Section B - Short Questions ($60 \times 1 = 60$ Marks)**Q1. Predict the output:**

```
1 d = {"a": 1, "b": 2}
2 d["a"] += 3
3 print(d)
```

Q2. Predict the final set:

```
1 s = {1,2,3}
2 s.add(2)
3 s.remove(3)
4 print(s)
```

Q3. Explain what happens:

```
1 s = set()
2 s.update([1,2,3,2])
3 print(s)
```

Q4. Infer the dictionary after deletion:

```
1 d = {"x":5, "y":9, "z":12}
2 del d["y"]
3 print(d)
```

Q5. Predict membership:

```
1 s = {"red", "blue", "green"}
2 print("yellow" in s)
```

Q6. Explain the set difference:

```
1 a = {1,2,3,4}
2 b = {3,4,5}
3 print(a - b)
```

Q7. Predict the keys printed:

```
1 d = {"a":1, "b":2, "c":3}
2 for k in d:
3     print(k)
```

Q8. Predict the updated dictionary:

```
1 d = {"a": 1, "b": {"x": 3}}
2 d["b"]["x"] += d["a"]
3 print(d)
```

Q9. Predict final set contents:

```
1 s = {1, 2, 3}
2 s.discard(4)
3 s.update({2, 5})
4 print(s)
```

Q10. Predict set logic:

```
1 a = {1,2,3}
2 b = {3,4}
3 print(a.symmetric_difference(b))
```

Q11. Explain ordering in iteration:

```
1 d = {"z":3, "a":1, "m":2}
2 print(list(d.keys()))
```

Q12. Explain membership rules:

```
1 s = {"hi", ("a",1)}
2 print(("a",1) in s)
```

Q13. Interpret nested update:

```
1 d = {"a":{"k":1}}
2 d["a"].update({"k":10})
3 print(d)
```

Q14. Predict the output:

```
1 a = np.array([1,2,3])
2 print(a*3)
```

Q15. Interpret slicing:

```
1 a = np.arange(10)
2 print(a[2:7:2])
```

Q16. Predict flattened output:

```
1 a = np.array([[1,2],[3,4]])
2 print(a.ravel())
```

Q17. Interpret broadcasting:

```
1 a = np.array([1,2,3])
2 b = np.array([[1],[2],[3]])
3 print(a + b)
```

Q18. Summarize the dtype behavior:

```
1 a = np.array([1.5, 2, 3])
2 print(a.dtype)
```

Q19. Interpret reshaping:

```
1 a = np.arange(6)
2 b = a.reshape(2,3)
3 print(b)
```

Q20. Interpret transpose:

```
1 a = np.arange(9).reshape(3,3)
2 print(a.T)
```

Q21. Predict stacking:

```
1 x = np.array([1,2])
2 y = np.array([3,4])
3 print(np.column_stack((x,y)))
```

Q22. Predict slicing + broadcasting:

```
1 import numpy as np
2 a = np.arange(10)
3 print(a[1:6] + 2)
```

Q23. Interpret filtering:

```
1 a = np.array([3,8,1,9])
2 print(a[a%2==1])
```

Q24. Infer reshape effect:

```
1 a = np.arange(9)
2 b = a.reshape(3,3)
3 print(b[1])
```

Q25. Explain ravel vs. flatten:

```
1 a = np.array([[1,2],[3,4]])
2 b = a.ravel()
3 b[0] = 99
4 print(a)
```

Q26. Predict type promotion:

```
1 a = np.array([1,2,3])
2 b = np.array([0.5])
3 print((a+b).dtype)
```

Q27. Convert a list of lists to array:

```
1 x = [[1,2],[3,4]]
2 print(np.array(x).ndim)
```

Q28. Predict random masking:

```
1 np.random.seed(0)
2 a = np.random.randint(1,10,5)
3 print(a[a>5])
```

Q29. Explain stack behavior:

```
1 x = np.array([1,2])
2 y = np.array([3,4])
3 print(np.vstack((x,y)))
```

Q30. Interpret the Series:

```
1 import pandas as pd
2 s = pd.Series([10,20,30],
3               index=["a","b","c"])
4 print(s["b"])
```

Q31. Predict DataFrame column:

```
1 df = pd.DataFrame({"x": [1,2], "y": [3,4]})
2 print(df["y"])
```

Q32. Explain index alignment:

```
1 s1 = pd.Series([1,2], index=["a","b"])
2 s2 = pd.Series([10,20], index=["b","c"])
3 print(s1 + s2)
```

Q33. Convert dict → DataFrame:

```
1 d = {"A": [1,2], "B": [3,4]}
2 print(pd.DataFrame(d))
```

Q34. Interpret filtering:

```
1 df = pd.DataFrame({"x": [1,4,6]})
2 print(df[df["x"]>3])
```

Q35. Interpret DataFrame shape:

```
1 df = pd.DataFrame({"a": [1,2,3],
2                    "b": [4,5,6]})
3 print(df.shape)
```


Q36. Predict index lookup:

```
1 s = pd.Series([10,20], index=["x","y"])
2 print(s.loc["y"])
```

Q37. Infer column addition:

```
1 df = pd.DataFrame({"a": [1,2], "b": [3,4]})
2 df["c"] = df["a"] + df["b"]
3 print(df)
```

Q38. Explain alignment:

```
1 s1 = pd.Series([1,2], index=["a","b"])
2 s2 = pd.Series([10], index=["b"])
3 print(s1 + s2)
```

Q39. Predict sorting:

```
1 df = pd.DataFrame({"x": [3,1,2]})
2 print(df.sort_values("x"))
```

Q40. Interpret loc slice:

```
1 df = pd.DataFrame({"v": [10,20,30]},
2                   index=["a","b","c"])
3 print(df.loc["b":])
```

Q41. Predict describe():

```
1 df = pd.DataFrame({"x": [5,10,15,20]})
2 print(df.describe().loc["mean"])
```

Q42. Predict the split:

```
1 s = "a,b,c"
2 print(s.split(","))
```

Q43. Interpret slicing:

```
1 s = "ABCDE"
2 print(s[1:4])
```

Q44. Explain case conversion:

```
1 s = "Hello"
2 print(s.lower())
```

Q45. Predict strip result:

```
1 s = "  test  "
2 print(s.strip())
```

Q46. Interpret count():

```
1 s = "banana"
2 print(s.count("an"))
```

Q47. Convert string → list:

```
1 s = "python"
2 print(list(s))
```

Q48. Predict join():

```
1 print("-".join(["a", "b", "c"]))
```

Q49. Interpret replace():

```
1 s = "2025-12-11"
2 print(s.replace("-", "/"))
```

Q50. Predict find():

```
1 s = "hello world"
2 print(s.find("world"))
```

Q51. Predict slicing:

```
1 s = "abcdef"
2 print(s[::2])
```

Q52. Explain replace:

```
1 s = "banana"
2 print(s.replace("a","o",2))
```

Q53. Predict splitting:

```
1 s = "a::b::c"
2 print(s.split("::"))
```

Q54. Infer find() behavior:

```
1 s = "hello world"
2 print(s.find("o",5))
```

Q55. Predict capitalization:

```
1 s = "python"
2 print(s.capitalize())
```

Q56. Predict count():

```
1 s = "aaaa"
2 print(s.count("aa"))
```

Q57. Infer strip():

```
1 s = "***hello***"
2 print(s.strip("*"))
```

Q58. Predict lines printed:

```
1 for line in ["a\n","b\n"]:
2     print(line.strip())
```

Q59. Explain exception flow:

```
1 try:
2     x = 10/0
3 except ZeroDivisionError:
4     print("error")
```

Q60. Predict printed lines:

```
1 for line in ["hi\n", "bye\n"]:
2     print(line.rstrip())
```

— End of Section —

Section C - Interpreting Python Code for Climate Data Analysis (10 marks)

Task: Carefully examine the Python script provided below. For each logically separated block of code, **interpret** its purpose in the overall workflow. In your own words, **explain** what each section is doing and why it is necessary for conducting a climate-trend analysis.

```
1
2 import pandas as pd
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5 from scipy.stats import linregress
6
7
8 df = pd.read_csv('nyc_january_avg_highs.csv', skiprows=2)
9 df.head()
10
11
12 df.columns = ['Date', 'Temperature']
13
14
15 df.info()
16
17 # Extract year from the 'Date' column (first 4 digits)
18 df["Year"] = df["Date"].astype(str).str[:4].astype(int)
19
20
21 df = df[["Year", "Temperature"]]
22 df.head()
23
24
25 sns.set_style("whitegrid")
26 plt.figure(figsize=(5,3))
```

```
27 sns.scatterplot(data=df, x="Year", y="Temperature")
28 plt.title("NYC Average January High Temperatures (1895{2025}")
29 plt.xlabel("Year")
30 plt.ylabel("Temperature (°F)")
31 plt.show()
32
33
34 slope, intercept, r_value, p_value, std_err = linregress(df["Year"], df["Temperature"])
35 print(f"Slope (m): {slope:.4f}")
36 print(f"Intercept (b): {intercept:.2f}")
37 print(f"R-squared: {r_value**2:.3f}")
38
39
40 plt.figure(figsize=(5,3))
41 sns.regplot(data=df, x="Year", y="Temperature", line_kws={"color":"red"})
42 plt.title("Trend of NYC January High Temperatures (1895{2025}")
43 plt.xlabel("Year")
44 plt.ylabel("Temperature (°F)")
45 plt.show()
46
47
48 year = 2026
49 predicted_temp = slope * year + intercept
50 print(f"Predicted January Avg High in {year}: {predicted_temp:.2f} °F")
51
52
53 pd.set_option('display.precision', 2)
54 df['Temperature'].describe()
```

— End of Paper —